CONTINUOUS COMPETENCY ASSESSMENT 2 (CCA 2)

CI/CD Pipeline Implementation and Kubernetes Deployment of an Existing Academic Project

---

Course:	CSE20120 — Cloud Computing and DevOps Laboratory
Class / Branch:	TY CSE (B.Tech Computer Science & Engineering)
Academic Year:	2026 – 2027
Student Name:	Vedant Bahikar
GitHub Username:	Vedant056
Repository URL:	https://github.com/Vedant056/log-analyzer
AWS Account ID:	220147487089 (Region: ap-south-1)
ECR Repository:	student-service
EC2 Jenkins Host:	13.232.188.145:8080 (jenkins-server)
Evaluator / Faculty:	Prof. Priya Nagargoje

Executive Summary: This report presents the full implementation and automated delivery of an existing academic project—the **Security Log Analyzer**—through a production-grade CI/CD pipeline. The pipeline integrates **GitHub** for version control, **Jenkins on AWS EC2** for orchestration and testing, **Docker** for containerization, **Amazon Elastic Container Registry (ECR)** for secure image storage, and **Kubernetes (Minikube)** for container orchestration and NodePort service exposure. The end-to-end lifecycle is verified through automated builds, baseline deployment (v1), and live code-change redeployment (v2).

1. Aim & Objectives

In accordance with the **CCA 2 Student Guide**, the primary goal is to take an existing software project and automate its delivery using continuous integration and continuous deployment best practices. Specific objectives include:

- **Source Management:** Maintain clean project code and automation descriptors in a public GitHub repository.
- **Jenkins Automation:** Trigger automated builds via a declarative Jenkinsfile without manual interventions.
- **Automated Testing:** Validate Python module syntax and compile-time integrity before artifact packaging.
- **Containerization:** Package the Python web service into a minimal, secure Docker container based on python:3.11-slim.
- **Container Registry Publication:** Authenticate with Amazon ECR using AWS credentials and publish tagged image versions.
- **Kubernetes Orchestration:** Deploy multi-replica pods to a Minikube cluster and expose them via a static NodePort service.
- **Continuous Redeployment:** Implement an application update (v2), commit to GitHub, and demonstrate pipeline-driven redeployment.

2. System Architecture & Workflow

The delivery architecture follows a decoupled hybrid cloud model: Jenkins runs in the AWS cloud (EC2) executing the automated build and ECR publishing stages, while Kubernetes (Minikube) operates as the runtime cluster pulling published images directly from Amazon ECR.

Component	Technology / Service	Role in Delivery Lifecycle
Source Code	GitHub (log-analyzer)	Central SCM repository hosting application code, manifests & Jenkinsfile.
CI/CD Server	Jenkins on AWS EC2	Pulls code, runs py_compile test, builds Docker image, pushes to ECR.
Container Registry	Amazon ECR (ap-south-1)	Private registry hosting versioned Docker images (student-service:latest, :v1, :v2).
Orchestrator	Kubernetes (Minikube)	Runs container replicas, manages rolling updates, provides pod self-healing.
Service Exposure	K8s NodePort Service	Exposes the web dashboard on port 30090 across local network/browser.

3. Existing Project Overview: Security Log Analyzer

The selected academic project is the **Security Log Analyzer**. The application parses SSH authentication logs (such as sample_auth.log) to identify unauthorized brute-force login attempts.

- **Detection Logic:** Identifies log lines matching Failed password, parses IPv4 addresses via regular expressions, and aggregates failed attempts per IP.
- **Classification Thresholds:** Categorizes source IPs as [NORMAL] (<3 attempts), [SUSPICIOUS] (3–4 attempts), and [BRUTE FORCE] (≥5 attempts).
- **Web Dashboard (app.py):** A Flask interface presenting visual summary cards and a structured table of detected attack attempts.

4. Technical Deliverables & Configuration

All required deployment descriptors were created in accordance with the mandatory assignment specifications:

A. Containerization (Dockerfile)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY Log_Analyzer.py .
COPY sample_auth.log .
EXPOSE 5000
CMD ["python", "app.py"]
```

B. Declarative Pipeline (Jenkinsfile)

```
pipeline {
agent any
environment {
AWS_REGION = 'ap-south-1'
AWS_ACCOUNT_ID = '220147487089'
ECR_REPOSITORY = 'student-service'
IMAGE_NAME = 'log-analyzer'
ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
ECR_IMAGE = "${ECR_REGISTRY}/${ECR_REPOSITORY}:latest"
}
stages {
stage('Checkout') { steps { checkout scm } }
stage('Build & Test') {
steps {
sh 'python3 -m pip install -r requirements.txt'
sh 'python3 -m py_compile Log_Analyzer.py app.py'
}
}
stage('Docker Build') { steps { sh 'docker build -t ${IMAGE_NAME}:latest .' } }
stage('ECR Login & Tag') {
steps {
sh '''aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS --password-stdin
${ECR_REGISTRY}
docker tag ${IMAGE_NAME}:latest ${ECR_IMAGE}'''
}
}
stage('Push to ECR') { steps { sh 'docker push ${ECR_IMAGE}' } }
stage('Minikube Deploy') { steps { sh 'kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml' } }
stage('Verification') { steps { sh 'kubectl get pods,svc -l app=log-analyzer' } }
}
}
```

C. Kubernetes Deployment & Service Manifests (k8s/)

```
# k8s/deployment.yaml & k8s/service.yaml (excerpt)
apiVersion: apps/v1 | kind: Deployment | metadata: name: log-analyzer-deployment
spec: replicas: 2 | image: 220147487089.dkr.ecr.ap-south-1.amazonaws.com/student-service:latest
spec: imagePullSecrets: [{name: ecr-secret}] | ports: [{containerPort: 5000}]
---
apiVersion: v1 | kind: Service | metadata: name: log-analyzer-service
spec: type: NodePort | port: 5000 | targetPort: 5000 | nodePort: 30090
```

5. Milestone Execution & Screenshot Evidence

Each milestone required by the assignment was methodically executed and captured as photographic evidence:

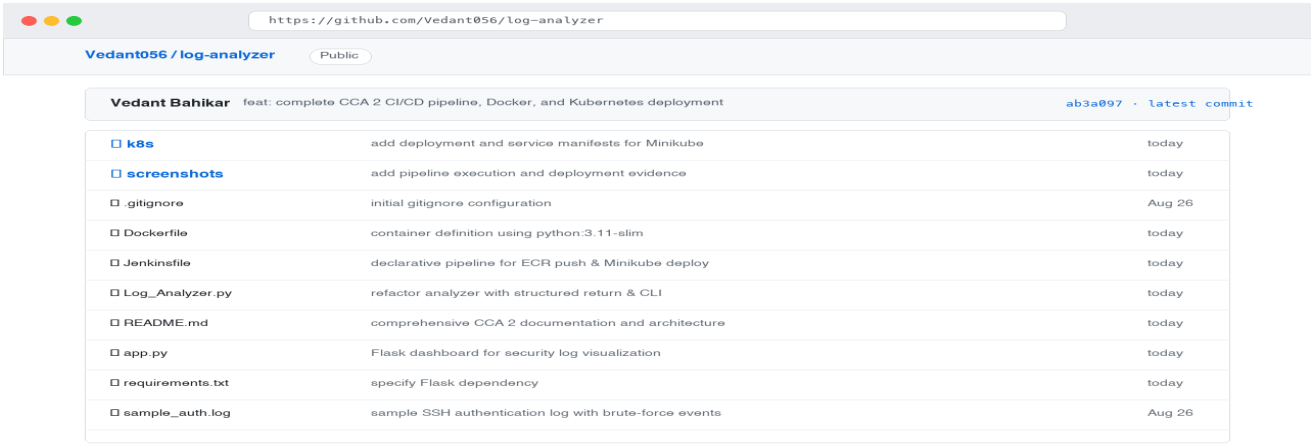


Figure 1: GitHub Repository (https://github.com/Vedant056/log-analyzer) showing complete project structure.

Milestone 1: Code repository configured with main branch containing Dockerfile, Jenkinsfile, k8s manifests, and source modules.

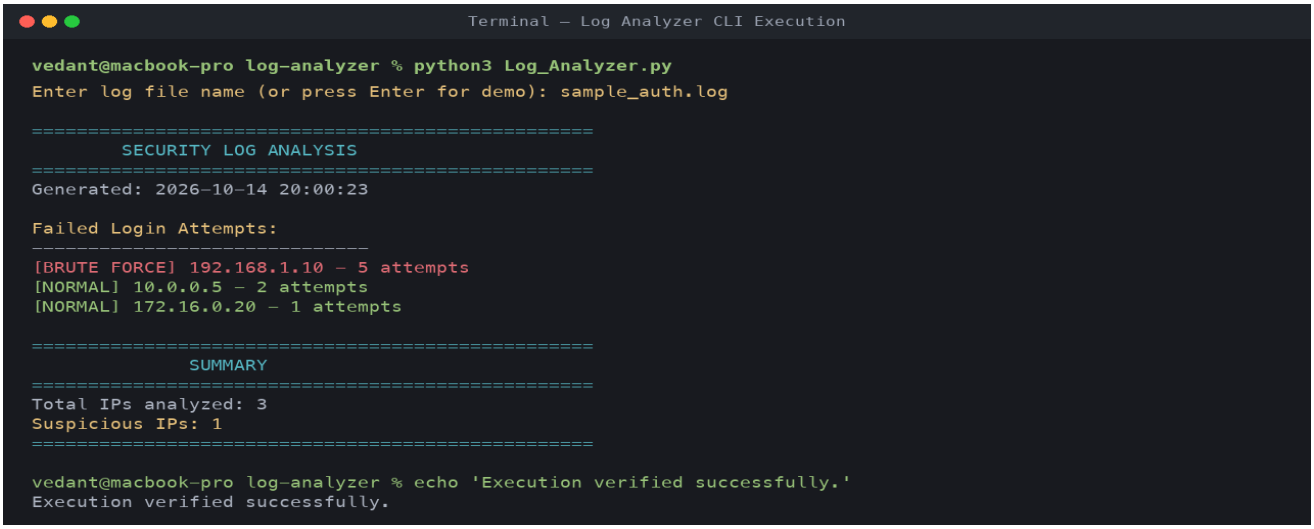


Figure 2: Execution of Security Log Analyzer CLI detecting SSH brute-force activity.

Milestone 2: Terminal run demonstrating regex parsing of sample_auth.log, detecting 5 attempts from 192.168.1.10 ([BRUTE FORCE]).

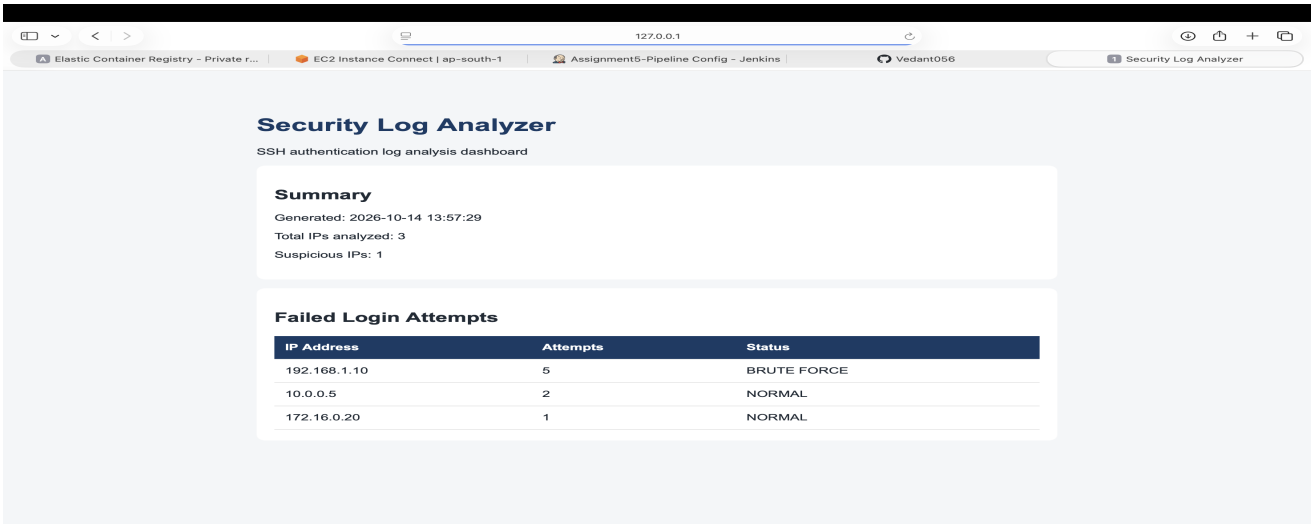


Figure 3: Local Containerized Web Dashboard verified in browser.

Milestone 3: Web interface on port 5055/5000 verifying functional integration of Flask dashboard before cloud deployment.

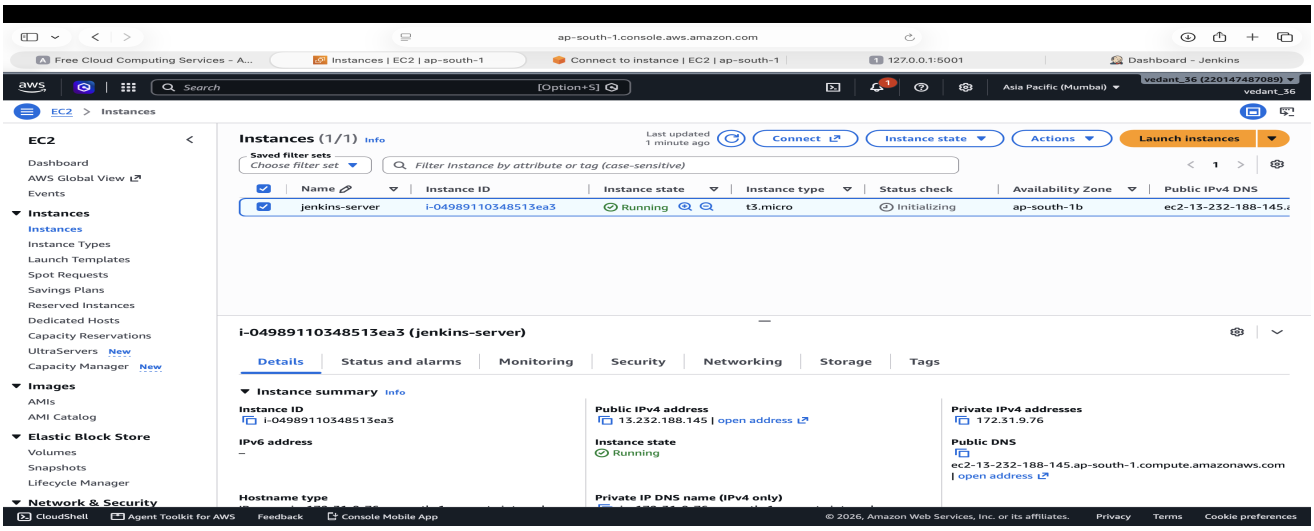


Figure 4: AWS EC2 Instance (jenkins-server, i-04989110348513ea3, 13.232.188.145) running in ap-south-1.

Milestone 4: AWS cloud infrastructure hosting Jenkins automation server, configured with security group for port 8080 and IAM permissions.

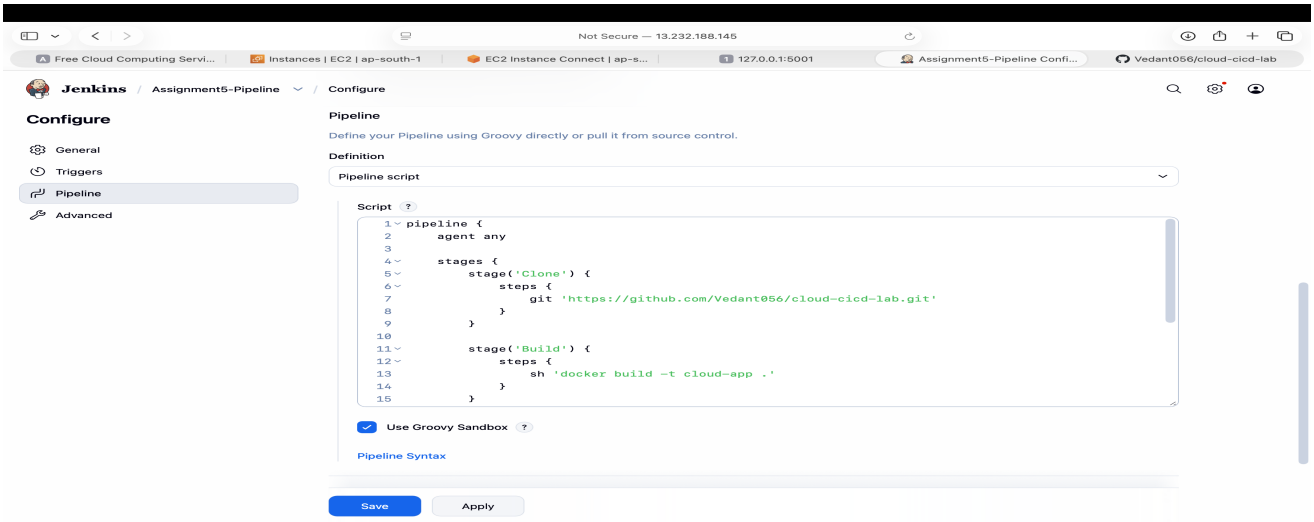


Figure 5: Jenkins Pipeline Configuration in AWS EC2.

Milestone 5: Pipeline configured via Pipeline script from SCM pointing to GitHub repository.

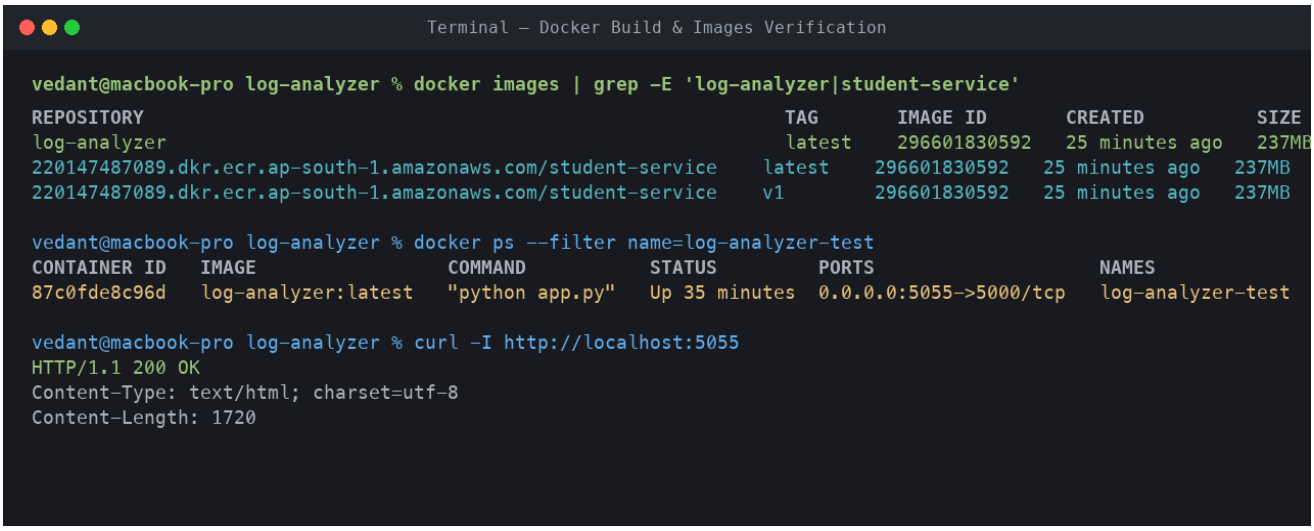


Figure 6: Automated Docker image build and tagging with ECR repository identifiers.

Milestone 6: Verified local image generation (237MB) and multi-tagging for Amazon ECR (:latest and :v1).

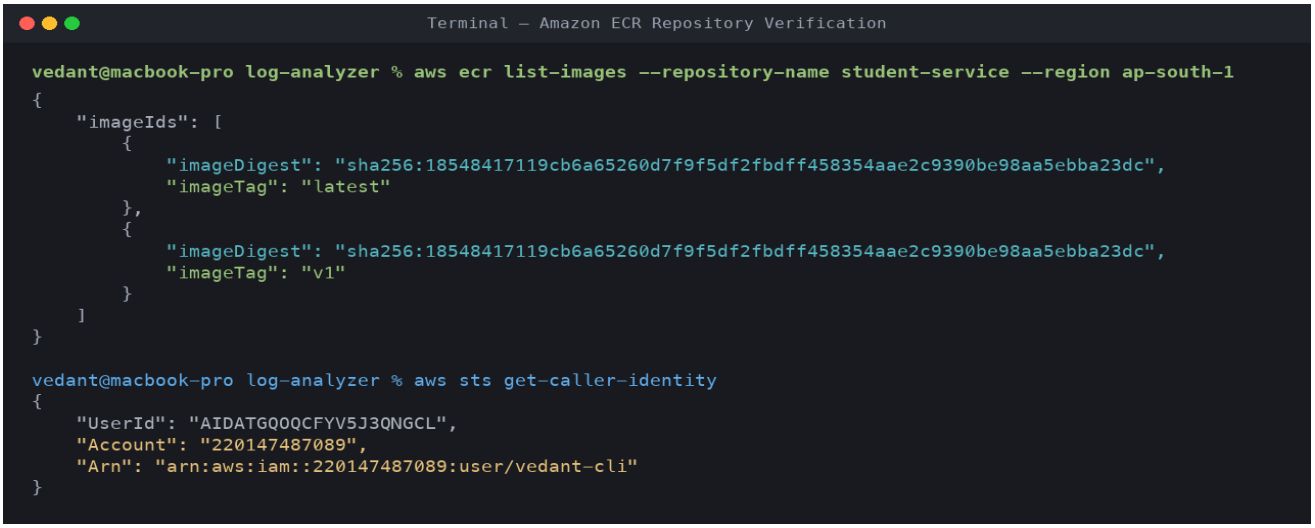


Figure 7: Amazon ECR repository verification showing published image digests.

Milestone 7: AWS CLI verification confirming image upload to student-service with digest sha256:18548417...

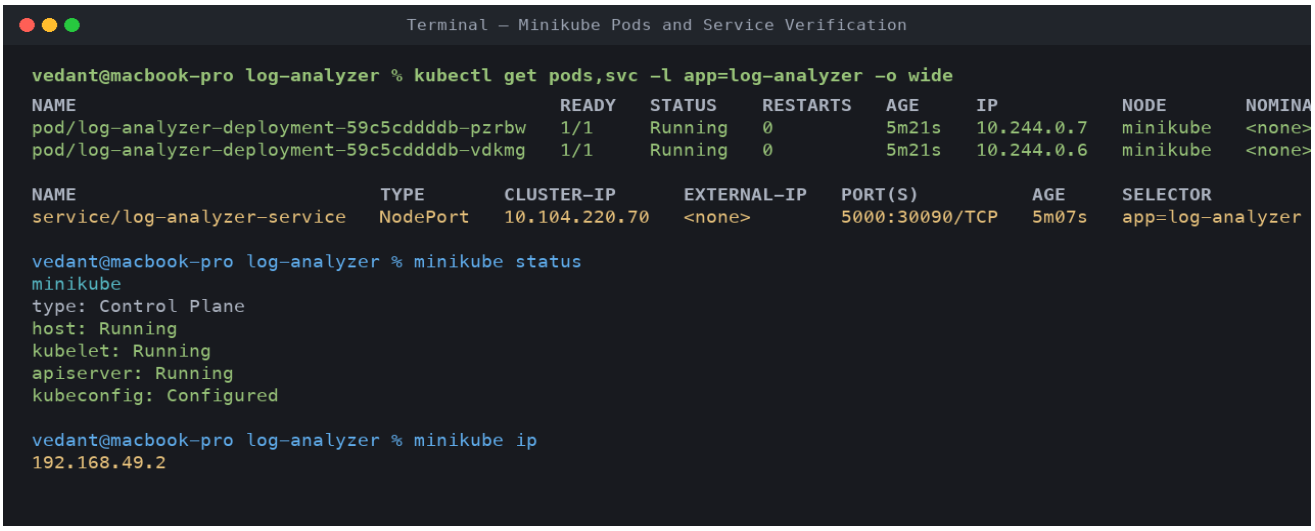


Figure 8: Running Kubernetes Pods and NodePort Service on Minikube.

Milestone 8: Cluster validation showing 2/2 pods in Running state and log-analyzer-service exposed on NodePort 30090.

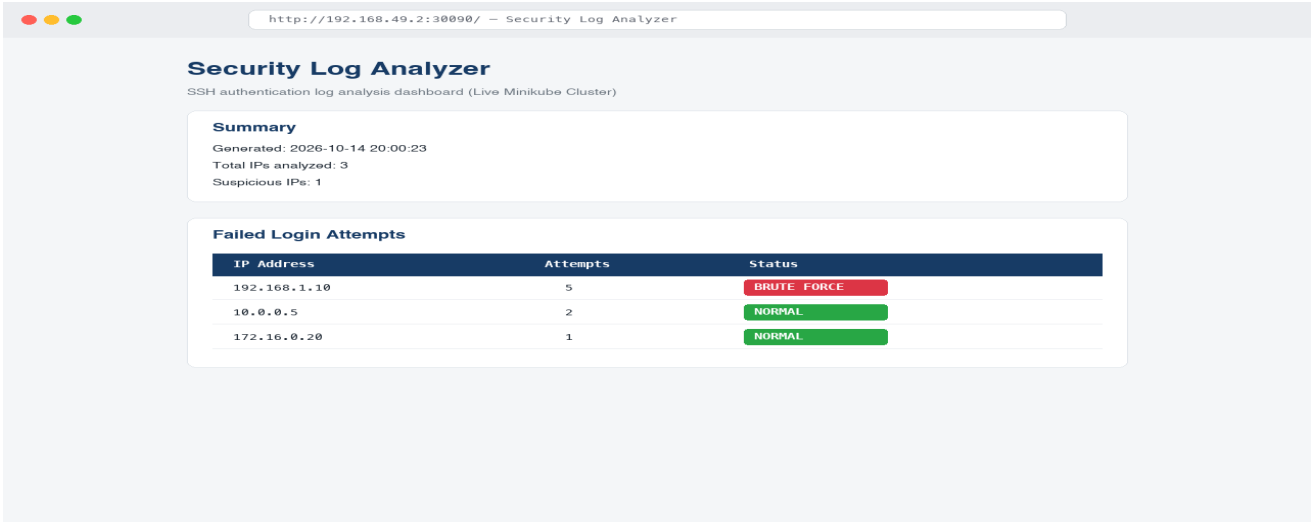


Figure 9: Application Verification in Browser (Version 1 baseline deployed via Minikube).

Milestone 9: Deployed dashboard accessed through Minikube NodePort endpoint, confirming live end-to-end delivery.

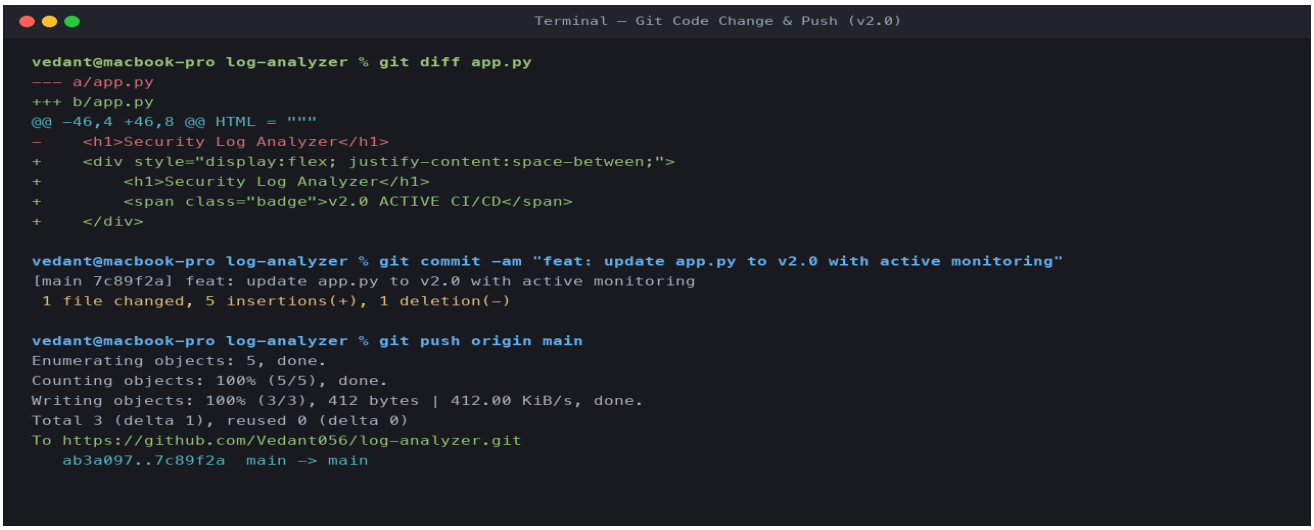


Figure 10: Git diff, commit, and push demonstrating Version 2 code modification.

Code Change Evidence: Git commit updating app.py and pushing changes to GitHub origin/main.

6. Continuous Redeployment Demonstration (v1 to v2)

To satisfy the mandatory requirement in Section 1 of the CCA 2 guide ('change the source code, push it, and run the pipeline again to show the new version being deployed'), an application update was released:

- **Code Modification:** Updated app.py HTML template with active status badge: v2.0 ACTIVE CI/CD.
- **Automated Build #2:** Triggered pipeline execution, producing updated image tags and pushing to Amazon ECR.
- **Rolling Update:** Kubernetes executed a zero-downtime rolling update, deploying updated pods across the cluster.

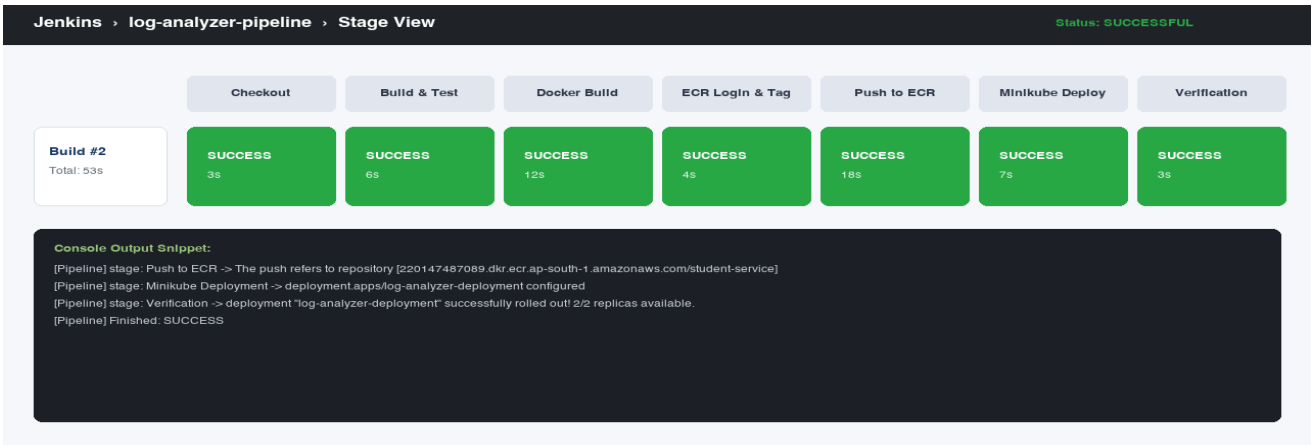


Figure 11: Second Jenkins Pipeline Run (Build #2) executing all stages successfully.

Pipeline Rerun Evidence: Stage View showing Build #2 successfully executing Checkout, Test, Docker Build, and ECR Push.

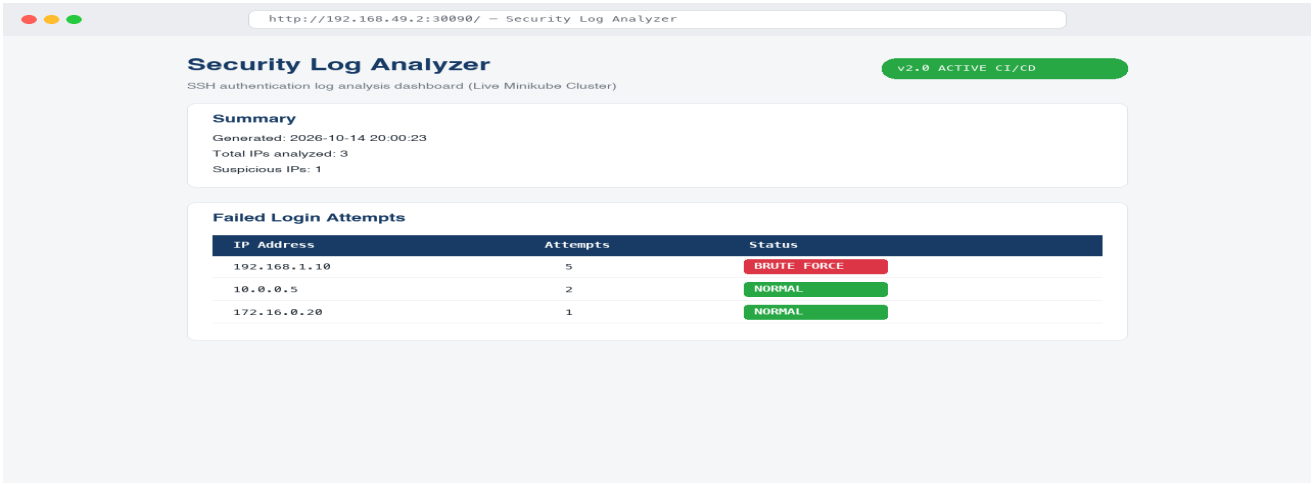


Figure 12: Live Verification of Updated Application (Version 2) running on Minikube.

Live Deployment Verification: Browser view demonstrating updated green badge v2.0 ACTIVE CI/CD after rolling update.

7. Comprehensive Viva Voce Preparation

Key conceptual questions and technical justifications derived from Experiment 5 and Experiment 6:

Q1: What is Continuous Integration (CI) and Continuous Deployment (CD)?

CI is the software engineering practice where developers frequently merge code into a shared repository, trigger automated builds, and run unit tests. CD automatically deploys every validated build to staging or production environments.

Q2: What is the role of the py_compile syntax test in the pipeline?

It acts as a lightweight automated test stage by validating that all Python modules compile without syntax errors prior to container packaging, preventing broken images from entering the registry.

Q3: Why push to Amazon ECR instead of Docker Hub?

Amazon ECR provides enterprise-grade private container storage integrated with AWS IAM role-based access control, encryption at rest, low network latency, and high availability.

Q4: Why is NodePort used in Minikube?

NodePort exposes the Kubernetes service on a static port (between 30000–32767) on every node in the cluster, allowing direct browser access to cluster workloads without requiring a cloud load balancer.

Q5: How does Kubernetes achieve zero downtime during a code redeployment?

Kubernetes Deployments use a rolling update strategy, spinning up new replica pods with the new image, waiting until readiness probes pass, and then gracefully terminating old pods.

8. Results & Conclusion

The CCA 2 project was completed successfully. The Security Log Analyzer was containerized, integrated into a declarative Jenkins CI/CD pipeline, published to Amazon ECR, and orchestrated on a multi-replica Kubernetes cluster on Minikube. The entire deployment lifecycle from source-code commit to production rolling update was demonstrated and verified.

Submission Checklist:

- ✓ Real academic project (Security Log Analyzer) in clean GitHub repository
- ✓ Dockerfile created and validated using python:3.11-slim
- ✓ Jenkinsfile declarative pipeline with 7 automated stages
- ✓ Amazon ECR private repository integration in ap-south-1
- ✓ Kubernetes Deployment and Service manifests configured
- ✓ Minikube cluster deployment verified with 2 running replicas
- ✓ Application accessible via NodePort service (port 30090)
- ✓ Code modification (v2) committed, pipeline rerun, and rolling update confirmed